

Communication & Performance Tutorial

Distributed CG on AMD MI300A — full tutorial

Communication & Performance Tutorial — Distributed CG on AMD GPUs

A step-by-step tutorial built around the two examples in this repository:

- **CG-CPU/** — a distributed, CPU-only Conjugate Gradient (CG) solver (reference implementation).
- **CG-GPU/** — the same solver ported to AMD GPUs (rocSPARSE / rocBLAS / RCCL) with **five interchangeable communication variants** for the halo exchange.

The goal is not just to run the examples, but to walk the **path to a trustworthy, optimized performance measurement** so you can fairly compare communication approaches on AMD Instinct MI300A hardware.

Who this is for

You are comfortable with C++/MPI and have access to an MI300A node (or similar CDNA3 APU) with ROCm and a GPU-Aware MPI. You want to understand how to measure GPU communication cost correctly and which transport to pick.

The path (read in order)

#	Chapter	What you get
0	Introduction & setup	The problem, the two examples, hardware/software prerequisites
1	The CPU reference	Build/run CG-CPU, understand the algorithm and distributed SpMV
2	Porting to the GPU	Build/run CG-GPU, what changed (rocSPARSE/rocBLAS/RCCL/GPU-Aware MPI)
3	Measuring performance correctly	The core chapter: affinity, reproducibility, warm-up, repeats, isolating comm vs compute
4	Comparing communication variants	The five transports, a fair comparison, and how to read the results
5	Tuning the copy engines (SDMA vs blit)	HSA_ENABLE_SDMA / _GANG, when each wins
6	ROCm version & the SpMV API	Version sweeps, the 7.x compute regression, rocsparse_v2_spmv, OpenMPI vs cray-mpich
7	The optimized configuration	Putting it together: a recommended launch, a checklist, and where to go next

TL;DR — the recommended optimized measurement

If you only remember one thing: **fix affinity and the RHS seed first, then take the minimum of several warm runs, and always split the reported time into communication vs. compute.** Everything else (which transport, which copy engine, which ROCm version) is a second-order comparison that is only meaningful once the measurement itself is under control.

```
cd CG-GPU
make
# reproducible, affinity-bound, comm/compute split reported:
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend
```

The two in-depth results write-ups this tutorial draws on:

- [CG-GPU/STUDY_REPORT.md](#) — AAC6 / MI300A Ubuntu, OpenMPI, ROCm 6.4.1–7.13.
 - [CG-GPU/STUDY_REPORT_PrgEnv-amd.md](#) — HPE Cray EX, cray-mpich, ROCm 7.0.3.
-

0. Introduction & setup

The problem

Both examples solve the same linear system $\mathbf{A} \mathbf{x} = \mathbf{b}$ with the **Conjugate Gradient (CG)** method, where $\mathbf{A}$ is a large sparse symmetric positive-definite (SPD) matrix. The test matrix `Dubcova2.pm` is $65\,536 \times 65\,536$ and SPD. The matrix is distributed across MPI ranks by rows; each rank owns a contiguous block of rows and the corresponding slice of the solution vector.

CG is an iterative solver. Each iteration needs:

1. one **sparse matrix-vector product** (SpMV), $\mathbf{A} \mathbf{p}$ — this is where inter-rank **communication** happens, because a rank needs "ghost" values of $\mathbf{p}$ owned by its neighbours;
2. two **dot products** ($\mathbf{r} \cdot \mathbf{r}$ and $\mathbf{p} \cdot \mathbf{A} \mathbf{p}$) — each requires an `MPI_Allreduce` across all ranks;
3. a few **vector updates** ($\mathbf{a} \mathbf{x} \mathbf{p}$, $\mathbf{s} \mathbf{c} \mathbf{a} \mathbf{l}$) — purely local.

So a distributed CG iteration has exactly two kinds of communication:

- **halo exchange** (neighbour-to-neighbour) inside the SpMV, and
- **global reductions** (`MPI_Allreduce`) for the dot products.

This tutorial is about measuring and optimizing the **halo exchange** — the part where you get to choose the transport — while keeping everything else fixed.

The two examples

```
cg-solver-example/  
├─ CG-CPU/      reference: plain C++/MPI, std::vector, hand-written loops (no GPU)  
└─ CG-GPU/      GPU port: rocSPARSE / rocBLAS / RCCL + GPU-Aware MPI, 5 comm variants
```

`CG-GPU/` is a line-by-line evolution of `CG-CPU/`: same algorithm, same matrix, same MPI comm pattern setup. Only the numerical kernels move to the GPU and the halo exchange gains four alternative implementations. That shared lineage is what makes the comparison meaningful.

The five communication variants (preview)

method	transport	buffers	collective?
staged	<code>MPI_Isend/Irecv</code>	host (D→H, H→D copies)	no
isend	<code>MPI_Isend/Irecv</code>	GPU (GPU-Aware MPI)	no
rccl	<code>ncclSend/ncclRecv</code>	GPU (RCCL)	no
alltoallv_staged	<code>MPI_Alltoallv</code>	host	yes
alltoallv	<code>MPI_Alltoallv</code>	GPU (GPU-Aware MPI)	yes

All five produce **identical numerical results** — they differ only in how ghost values move.

Hardware & software prerequisites

Hardware: an AMD Instinct **MI300A** node (this tutorial's reference). MI300A is an APU: CPU + GPU share HBM, and each of the 4 (or 8, depending on partitioning) GPU dies is local to one NUMA node. The affinity lessons in Chapter 3 apply to any multi-GPU AMD node.

Software:

- **ROCm** ≥ 6.3 (provides `hipcc`, `rocSPARSE`, `rocBLAS`, `RCCL`). This tutorial shows results across 6.4.1–7.13.
- A **GPU-Aware MPI**: either OpenMPI/UCX built `--with-rocm`, or HPE `cray-mpich` with `craype-accel-amd-gfx942`.
- A batch scheduler (SLURM) for reproducible, exclusive allocations.

Two module setups appear throughout:

```
# OpenMPI + ROCm (AAC6 / MI300A Ubuntu)
module load rocm/6.4.1
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4

# HPE Cray EX + cray-mpich (PrgEnv-amd)
module swap PrgEnv-cray PrgEnv-amd/8.7.0
module load craype-accel-amd-gfx942
```

The `CG-GPU/Makefile` auto-detects which of these you are in (it keys off the Cray `PE_ENV`), so `make` works in both without edits.

What "done" looks like

By the end you will be able to produce a table like this — and, more importantly, **trust it**:

method	solve (s)	comm (s)	compute (s)	comm %
staged	0.148	0.124	0.023	84 %
isend	0.051	0.028	0.023	55 %
rccl	0.051	0.025	0.025	50 %
alltoallv_staged	0.082	0.056	0.026	68 %
alltoallv	0.051	0.025	0.026	49 %

(AAC6, ROCm 6.4.1, 4 ranks, `CG_SEED=12345`. Your absolute numbers will differ; the methodology is what transfers.)

Next: [1. The CPU reference →](#)

1. The CPU reference (CG-CPU/)

Start here even if you only care about the GPU. The CPU version is small, has no GPU or math-library moving parts, and establishes the algorithm and the distributed communication pattern that the GPU version inherits unchanged.

Build & run

```
cd CG-CPU
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4 # any MPI with mpicxx
make
mpirun -n 4 ./cg_cpu src/Dubcova2.pm
```

Expected output:

```
172 iterations to converge
2-norm of residual: 0.000190
CG solve time:      0.1588 s (0.0009 s/iter)
```

The iteration count and residual will wobble slightly run-to-run because the right-hand side `b` is built from a **random** `x0`. Hold that thought — reproducibility is the subject of Chapter 3.

What's in `cg.cpp`

function	role
<code>spmv(Mat&, ...)</code>	serial CSR sparse matrix-vector product (single block)
<code>spmv(ParMat&, ...)</code>	distributed SpMV: post receives, pack+send, on-proc SpMV, wait, off-proc SpMV
<code>inner_product</code>	local dot product + <code>MPI_Allreduce</code>
<code>axpy / scale</code>	$x \mathrel{+}= \alpha y$ / $x \mathrel{*}= \alpha$ (local loops)
<code>main</code>	read matrix, form comm pattern, run CG

The distributed SpMV — the pattern that matters

This is the halo exchange, and its structure is identical in every GPU variant later:

1. Post non-blocking receives for ghost values (MPI_Irecv)
2. Pack and post non-blocking sends (loop + MPI_Isend)
3. Compute the on-proc SpMV (overlaps with in-flight messages)
4. Wait for receives (MPI_Waitall)
5. Compute the off-proc SpMV using the ghosts
6. Wait for sends (MPI_Waitall)

Two ideas are baked in here and carry through the whole tutorial:

- **The matrix is split into an on-proc block and an off-proc block.** The on-proc block only touches locally owned vector entries, so it can be computed while messages are in flight. The off-proc block needs the ghost values, so it runs after the receives complete. This on-proc/off-proc split is what makes communication/computation **overlap** possible.
- **The comm pattern is set up once.** `sparse_mat.hpp` (`form_comm`) figures out, per rank, exactly which indices to send to which neighbour and how many to receive. This setup is done on the CPU and is identical in `CG-GPU/` — only the data movement changes.

The CG algorithm

```
r0 = b - A x0
p0 = r0
for i = 0, 1, 2, ...:
    α = (ri, ri) / (Api, pi)      ← needs Api (SpMV) and two dot products
    x += α pi
    r = b - A x                    ← residual recomputed every 8 iters for stability
    β = (ri+1, ri+1) / (ri, ri)
    p = r + β p
```

Per iteration: **1 SpMV** (halo exchange) + **2 dot products** (`MPI_Allreduce`) + a few local vector ops. That fixed structure is the "denominator" of every measurement later — the work per iteration never changes, so if the iteration count is held constant (Chapter 3), differences in solve time come only from the transport and the hardware/software stack.

Why keep the CPU version around?

- It is the **correctness oracle**: the GPU variants must converge to the same residual in the same number of iterations (for a fixed seed).
- It is a **baseline** for "how much did the GPU actually buy us?" — and a reminder that for a small per-rank problem the answer is subtle (the CPU solve here, ~0.16 s, is in the same ballpark as some unoptimized GPU launches, which is exactly why measurement discipline matters).

Next: [2. Porting to the GPU →](#)

2. Porting to the GPU (CG-GPU/)

CG-GPU/ keeps the algorithm and the MPI comm-pattern setup from CG-CPU/ and moves every numerical operation to the GPU. The two headers (`sparse_mat.hpp`, `par_binary_IO.hpp`) are the same — file I/O and comm setup stay on the host.

What changed

CG-CPU/	CG-GPU/
<code>std::vector<double></code>	<code>hipMalloc / hipMemcpy</code> device arrays
hand-written SpMV loop	<code>rocsparse_spmv</code> (rocSPARSE generic API)
hand-written pack/gather	<code>rocsparse_dgthr</code> (GPU gather)
hand-written dot	<code>rocblas_ddot</code>
hand-written axpy/scal	<code>rocblas_daxpy / rocblas_dscal</code>
MPI_Isend/Irecv on host buffers	choice of 5 communication variants

Build & run

```
cd CG-GPU
module load rocm/6.4.1
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
make
mpirun -n 4 ./cg_gpu src/Dubcova2.pm isend
```

The `Makefile` auto-detects the toolchain (OpenMPI+ROCM vs Cray PE) and the GPU arch. Useful knobs:

```
make GPU_ARCH=gfx90a    # target a specific arch (default: auto-detected, fallback gfx942)
make SPMV_V2=1          # use rocsparse_v2_spmv (ROCM 7.x); see Chapter 6
make run                 # quick 4-rank sanity run over four methods
```

A first run prints something like:

```
method=isend ranks=4 gpus_visible=1
RHS seed: 12345 (CG_SEED)
Initial residual: 195.873
174 iterations to converge
2-norm of residual: 0.000188555
CG solve time:      0.0513 s (0.0003 s/iter)
  comm total:      0.0282 s (55.0% of solve)
  halo exchange:   0.0142 s (0.0001 s/iter, 27.6%)
  dot allreduce:   0.0140 s (27.3%)
  compute (rest):  0.0231 s (45.0%)
```

That last block — the **communication breakdown** — is instrumentation added for this tutorial and is the key to everything that follows. We'll dissect it in Chapter 3.

The three ports you should actually look at

1. SpMV: a library call instead of a loop

```
// CPU:
for (int i = 0; i < A.n_rows; i++)
    for (int j = A.rowptr[i]; j < A.rowptr[i+1]; j++)
        b[i] += alpha * A.data[j] * x[A.col_idx[j]];

// GPU:
rocsparse_spmv(handle, rocsparse_operation_none,
               &alpha, A.descr, vec_x, &beta, vec_b, ...);
```

The rocSPARSE descriptor wraps the CSR arrays uploaded once with `hipMemcpy`. The on-proc and off-proc blocks each get their own descriptor, preserving the overlap structure from Chapter 1.

2. Halo exchange: the send buffer is packed on the GPU

```
// Pack the send buffer on the GPU – no host copy
rocsparse_dgthr(handle, send_size, d_x, d_sendbuf, d_send_idx, ...);
hipDeviceSynchronize();           // gather must finish before MPI reads d_sendbuf

// GPU-Aware MPI: hand GPU pointers straight to MPI
MPI_Isend(d_sendbuf + off, count, MPI_DOUBLE, dest, ...);
MPI_Irecv(d_recvbuf + off, count, MPI_DOUBLE, src, ...);
```

Without GPU-Aware MPI you would copy device→host before the send and host→device after the receive. The `staged` variant does exactly those copies; the `isend` variant skips them. Comparing the two (Chapter 4) is how you measure the value of GPU-Aware MPI.

3. Dot products: `rocbblas_ddot` then one sync then `MPI_Allreduce`

```
rocbblas_ddot(blas_handle, n, d_a, 1, d_b, 1, &local_sum);
hipDeviceSynchronize();           // make the scalar visible on the host
MPI_Allreduce(&local_sum, &global, 1, MPI_DOUBLE, MPI_SUM, comm);
```

Every dot product forces a device→host sync. Two dots per iteration means two synchronization points per iteration — a latency cost that shows up as `dot allreduce` in the breakdown and is a prime optimization target (Chapter 7).

One GPU per rank

```
hipSetDevice(rank % num_gpus);
```

Under `mpirun` this hands ranks 0–3 distinct GPUs. That is correct but not the whole story: **which** cores a rank runs on relative to its GPU is what makes or breaks performance. That is Chapter 3, and it is the single most important chapter in this tutorial.

Next: [3. Measuring performance correctly →](#)

3. Measuring performance correctly

This is the heart of the tutorial. Before you compare any two communication approaches, the measurement itself has to be trustworthy. Five things, in priority order:

1. **Affinity** — bind each rank to its GPU-local CPU cores. (Biggest effect by far.)
2. **Reproducibility** — fix the RHS seed so every run solves the same system.
3. **Warm-up & repeats** — discard the first run; report the minimum (or median) of several.
4. **Isolate comm vs compute** — never compare a single "solve time" number blind.
5. **Verify correctness** — same iteration count and residual, or the comparison is meaningless.

Get these wrong and you will "measure" the scheduler, the RNG, or a cold cache instead of the transport.

3.1 Affinity — the make-or-break factor

On MI300A each GPU is local to one NUMA node (**GPU i $\leftrightarrow$ NUMA node i**). If a rank runs on cores that are remote from its GPU — or worse, if all ranks are crammed onto a handful of shared threads — every kernel launch, `hipDeviceSynchronize`, and MPI progress call pays a latency penalty.

How bad? On AAC6, the exact same binary and matrix, 4 ranks:

launch	CPU affinity	staged	isend	rccl	alltoallv
<code>--bind-to none, not exclusive</code>	4 hwthreads shared by all ranks (remote)	9.27 s	10.58 s	7.39 s	8.46 s
<code>--exclusive + gpu_bind.sh</code>	each rank $\rightarrow$ its GPU-local NUMA (24 cores)	0.15 s	0.05 s	0.05 s	0.05 s

That is a **$\sim 100\times$ difference** — dwarfing every transport, copy-engine, or ROCm-version effect in the rest of this tutorial. **If you take away one thing, take this: fix affinity first.**

Do it with a wrapper

Two ready-made per-rank wrappers ship with `CG-GPU/`:

```
# numactl-based, 4-GPU NUMA binding (OpenMPI):
mpirun -n 4 ./gpu_bind.sh ./cg_gpu_src/Dubcova2.pm isend

# taskset-based, 8-GPU MI300A binding (used by run_test_7.13.sh):
mpirun -n 8 --bind-to none bash set_affinity_mi300a.sh ./cg_gpu_src/Dubcova2.pm isend
```

Both set `ROCR_VISIBLE_DEVICES=<local_rank>` (each rank sees exactly its own GPU) and pin the rank to that GPU's local cores (`gpu_bind.sh` via `numactl --cpunodebind/--membind`; `set_affinity_mi300a.sh` via `taskset`). Use `--bind-to none` on the `mpirun/srun` side so the wrapper owns all pinning.

Verify it, don't assume it

```
mpicxx -O2 -std=c++17 check_affinity.cpp -o check_affinity \
-I$ROCM_PATH/include -L$ROCM_PATH/lib -lamdhip64
mpirun -n 4 ./check_affinity
```

This prints each rank's selected GPU **PCI bus id** and CPU mask and flags any two ranks that collide on a GPU.

Two affinity traps

- **srun without PMIx** launches each task as its own MPI world of size 1, so every task computes `rank % num_gpus = 0` and they **all pile onto GPU 0**. Use `srun --mpi=pmix` or a binding wrapper.
- **The cliff is a property of the launch, not the machine.** On an HPE Cray EX system an `--exclusive srun` allocation already binds each rank to a distinct, largely NUMA-local core set, so explicit binding adds only **$\sim 3\text{--}8\%$** there (see [STUDY_REPORT_PrgEnv-amd.md](#) §3). Never assume; measure bound vs. unbound on your system once, then standardize on the bound launch.

3.2 Reproducibility — fix the RHS seed

By default the right-hand side is seeded from wall-clock time, so each run solves a slightly different system and converges in a different number of iterations. You cannot compare methods whose iteration counts differ.

Fix the seed:

```
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend # via env
mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend 12345 # via argv[3]
```

The seed is echoed as `RHS seed: 12345` at the top of the output. With it fixed, **all five methods converge in exactly the same iteration count with the same residual** (e.g. 172 iterations, final residual 1.815e-4). Now a difference in solve time is a difference in transport, not in how much work was done.

3.3 Warm-up and repeats

The first run of a freshly built binary pays one-time costs: library initialization, JIT/code-object load, allocator warm-up, and RCCL communicator bring-up. These are not what you want to compare.

- **Discard the first run** (or treat it as warm-up).
- **Repeat 3-5 times** and report the **minimum** (cleanest, least noise) — or the **median $\pm$ spread** if you want to characterize variability. For the fast GPU-Aware variants the run-to-run spread is comparable to the method-to-method difference, so a single run can rank two methods backwards.

The sweep scripts already do this. For example `sweep_sdma.sh` keeps the minimum-solve run over `REPEATS`:

```
REPEATS=5 CG_SEED=12345 ... # every sweep script honours these
```

3.4 Isolate communication from compute

A single "solve time" number hides the thing you are trying to measure. The instrumented solver reports a breakdown (max across ranks, accumulated over the CG loop only):

```
CG solve time:      0.0513 s (0.0003 s/iter)
comm total:        0.0282 s (55.0% of solve)
halo exchange:     0.0142 s (0.0001 s/iter, 27.6%) ← the part you choose the transport for
dot allreduce:     0.0140 s (27.3%) ← MPI_Allreduce for the two dot products
compute (rest):    0.0231 s (45.0%) ← rocSPARSE SpMV + rocBLAS + launch overhead
```

- **halo exchange** = staging copies (D $\leftrightarrow$ H) + GPU gather/pack + the MPI/RCCL `send/recv/wait/alltoallv` calls. This is what differs between the five variants.
- **dot allreduce** = the global reductions. Independent of the halo transport.
- **compute (rest)** = `solve - comm total`. rocSPARSE/rocBLAS kernels + launch latency. Independent of the halo transport — so if it moves when you change only the transport, that is your noise floor talking.

Two definitions and a caveat you must keep in mind:

- The timers cover the **CG loop only** — matrix read, GPU upload, and setup are excluded (they are one-time).
- For the **overlap variants** (`isend`, `rccl`, `alltoallv`) the halo exchange figure is mostly the exposed wait: the on-proc SpMV runs concurrently with the in-flight messages, so this is **not** the raw wire time. It is the cost you actually pay, which is what you want for choosing a transport — just don't mistake it for a bandwidth measurement.

3.5 Verify correctness every time

Before trusting a timing, check the run actually solved the problem: same **iteration count** and **final residual** as the CPU reference (for the same seed). The bundled `test_cg_gpu.sh` does this automatically — it builds, runs every variant, asserts convergence to a tolerance, and prints the timing summary; it exits 0 gracefully on a node with no GPU.

```
# inside a GPU allocation, with binding:
GPU_BIND=./gpu_bind.sh CG_SEED=12345 ./test_cg_gpu.sh
# or submit:
sbatch submit_cg_gpu_test.sbatch
```

The measurement recipe (checklist)

```
[ ] Exclusive allocation (own whole NUMA nodes)
[ ] Per-rank GPU+CPU binding wrapper (gpu_bind.sh / set_affinity_mi300a.sh)
[ ] Verified binding once with check_affinity
[ ] Fixed CG_SEED
[ ] Warm-up run discarded
[ ] 3–5 repeats, report min (or median  $\pm$  spread)
[ ] Read the comm/compute breakdown, not just solve time
[ ] Confirmed identical iteration count + residual across methods
```

Only once every box is ticked are the comparisons in the next chapters meaningful.

Next: [4. Comparing communication variants](#) →

4. Comparing communication variants

With the measurement under control (Chapter 3), you can now compare the five transports fairly. The design of the examples makes this a set of clean, controlled experiments: pairs of variants that differ in exactly one thing.

The five variants and what each isolates

method	transport	buffers	pairs with...	to isolate
staged	Isend/Irecv	host	isend	the D→H staging copies
isend	Isend/Irecv	GPU (GPU-Aware)	staged	value of GPU-Aware point-to-point
alltoallv_staged	Alltoallv	host	alltoallv	the D→H staging copies (collective)
alltoallv	Alltoallv	GPU (GPU-Aware)	alltoallv_staged	value of GPU-Aware collective
rccl	ncclSend/Recv	GPU (RCCL)	isend/alltoallv	GPU-native vs GPU-Aware MPI

Two controlled contrasts fall out immediately:

- **staged vs isend** and **alltoallv_staged vs alltoallv** each isolate the **PCIe/staging round-trip** (device→host before the send, host→device after the receive) that GPU-Aware MPI removes.
- **isend vs alltoallv vs rccl** compares three GPU-resident transports head-to-head.

Run the comparison

```
cd CG-GPU && make
for m in staged isend rccl alltoallv_staged alltoallv; do
  echo "=== $m ==="
  CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm $m
done
```

Or use the harness that builds once and sweeps all methods with warm-up + repeats:

```
sbatch submit_cg_gpu_test.sbatch          # or: GPU_BIND=./gpu_bind.sh ./test_cg_gpu.sh
```

Results (AAC6, ROCm 6.4.1, 4 ranks, seed 12345, min of runs)

Times are seconds, max across ranks, CG loop only:

method	solve	comm total	halo exch	dot allreduce	compute	comm %
staged	0.1477	0.1243	0.1095	0.0148	0.0234	84 %
isend	0.0513	0.0282	0.0142	0.0140	0.0231	55 %
rccl	0.0505	0.0252	0.0160	0.0092	0.0253	50 %
alltoallv_staged	0.0823	0.0560	0.0422	0.0138	0.0263	68 %
alltoallv	0.0512	0.0250	0.0129	0.0121	0.0262	49 %

How to read it

- **Staging is the dominant cost when present.** staged halo (0.110 s) vs isend (0.014 s) is a ~7× gap that is purely the D→H + H→D copies GPU-Aware MPI eliminates. alltoallv_staged (0.042 s) vs alltoallv (0.013 s) shows the same round-trip on the collective path. **Takeaway: if you have GPU-Aware MPI, use it — this is the single largest transport-level win.**
- **The three GPU-resident transports are within noise of each other** here (~0.013–0.016 s halo). On one node, 4 GPUs, over XGMI, isend, alltoallv, and rccl have no decisive winner. Do not over-interpret a 1 ms gap that is smaller than your run-to-run spread — this is exactly why Chapter 3 insists on repeats.
- **compute is essentially constant across methods** (~0.023–0.026 s). Good — it should be, because you changed only the transport. If your compute column moves a lot between methods, your measurement is noisy (revisit affinity/warm-up), not the transport.

- **dot allreduce is a real, transport-independent cost** (0.009–0.015 s, up to ~27 % of solve): two latency-bound scalar allreduces per iteration, each gated by a `hipDeviceSynchronize`. No choice of halo transport touches it — it is a separate optimization axis (Chapter 7).

The subtlety: overlap means "halo" is exposed wait, not wire time

For `isend/rccl/alltoallv` the on-proc SpMV runs while messages are in flight, so the measured `halo exchange` is the part of the transfer that couldn't be hidden behind compute. That is the right number for "which transport costs me less," but it means:

- a faster raw transport can show a larger exposed halo if it leaves less compute to overlap with, and
- at larger per-GPU problem sizes (more on-proc work to hide behind), the exposed halo of the overlap variants shrinks further relative to `staged`.

To measure raw wire time instead, you would disable overlap or use a profiler trace (Chapter 7).

Does the ranking hold on a different MPI stack?

Yes. On HPE Cray EX with **cray-mpich** (ROCm 7.0.3), the same qualitative picture holds — staging is expensive, the GPU-resident transports are comparable, and `cray-mpich`'s GPU-Aware path is on par with (marginally better than) GPU-Aware OpenMPI:

method	solve	comm total	halo exch	dot allreduce	compute	comm %
staged	0.1722	0.1064	0.0862	0.0202	0.0657	62 %
isend	0.0973	0.0182	0.0146	0.0036	0.0791	19 %
rccl	0.0926	0.0129	0.0099	0.0030	0.0796	14 %
alltoallv_staged	0.1261	0.0547	0.0439	0.0107	0.0714	43 %
alltoallv	0.0971	0.0191	0.0156	0.0035	0.0779	20 %

(from [STUDY_REPORT_PrgEnv-amd.md](#) §4). Note the `comm %` is much lower here — not because communication got faster, but because compute got slower on ROCm 7.x. That cross-cutting effect is Chapter 6.)

Caveats

Single node, single matrix, single rank count, latency-bound regime. Absolute deltas between the fast variants are small; treat cross-method differences as **indicative**, and always report the measurement conditions with the numbers. Strong/weak scaling and larger per-GPU sizes (where a transport's bandwidth, not its latency, dominates) are the natural next experiments.

Next: [5. Tuning the copy engines \(SDMA vs blit\) →](#)

5. Tuning the copy engines (SDMA vs blit kernels)

Once you've chosen a GPU-Aware transport, there's a lower-level knob that changes how ROCm physically moves bytes between GPU memory and the fabric — and it can shift communication time by 20–30 % with **zero code changes**.

The two mechanisms

When GPU-Aware MPI (or the staging `hipMemcpy`) moves data, ROCm can use either:

mechanism	HSA_ENABLE_SDMA	how it works
SDMA engines	1 (default)	dedicated hardware DMA controllers move data independently of the shader engines — no compute resources consumed
Blit kernels	0	ROCm dispatches a small compute shader ("blit") that copies using the shader engines — no dedicated DMA hardware

A second flag gangs multiple SDMA engines onto one transfer:

```
export HSA_ENABLE_SDMA=1
export HSA_ENABLE_SDMA_GANG=1
```

Because this is purely a runtime setting, **the binary is identical across configurations** — you only change the environment. That makes it a clean controlled experiment: anything that moves is the copy path, and **compute should stay flat** (a built-in sanity check).

Run the sweep

Switch it by hand:

```
HSA_ENABLE_SDMA=1 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend # SDMA (default)
HSA_ENABLE_SDMA=0 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend # blit kernels
```

...or use the sweep harness (builds once, sweeps blit / sdma / sdma+gang across all methods, min of REPEATS):

```
sbatch --export=ALL,ROCM_VER=6.4.3,REPEATS=5 submit_sdma_sweep.sbatch
```

The maintainer's `run_test_7.13.sh` sweeps `HSA_ENABLE_SDMA ∈ {1,0}` for the GPU-Aware variants (`isend`, `alltoallv`) and labels each run (`sdma`) / (`blit_kernel`) in the log.

Results (ROCm 6.4.3, 4 ranks, seed 12345, min of 5 runs)

Halo-exchange time and its change vs. blit:

method	blit	sdma	sdma_gang	halo Δ vs blit
staged	0.1215	0.1116	0.1129	−8 %
isend	0.0154	0.0119	0.0113	−27 %
rccl	0.0143	0.0129	0.0129	−10 %
alltoallv_staged	0.0480	0.0418	0.0394	−18 %
alltoallv	0.0116	0.0140	0.0121	+21 % (see below)

(full solve/comm/compute table in [STUDY_REPORT.md](#) §5.2. The Cray/ROCm 7.0.3 sweep in [STUDY_REPORT_PrgEnv-amd.md](#) §5 shows the same pattern: `isend` −24 %, `alltoallv` −23 %.)

How to read it

- **SDMA helps the copy-heavy paths**, exactly as expected. Disabling it (blit) is slower on `staged` (−8 %), `alltoallv_staged` (−13...−18 %), and GPU-Aware `isend` (−23...−27 %). Blit copies burn CU/shader cycles and contend with the SpMV kernels; SDMA offloads them to dedicated hardware.

- **Ganging adds a small extra win** on the copy-bound variants (`isend` -23 %→-27 %, `alltoallv_staged` -13 %→-18 %) and is within noise elsewhere.
- **Compute stays flat** across all three settings — the effect is cleanly isolated to data transport, which is the confirmation that you're measuring the copy path and not something else.
- **The one exception** (`alltoallv` marginally better with blit) shows the general rule: for very small, latency-sensitive transfers, the SDMA engine's start-up cost can exceed a quick shader copy, especially on MI300A's unified memory. The effect is small and near the noise floor.

When does each win?

- **SDMA** wins for **larger / copy-heavy** transfers where keeping the shader engines free to overlap with compute matters more than raw copy latency.
- **Blit kernels** can win for **small, latency-sensitive** transfers where the shaders are otherwise idle and the lower dispatch overhead beats the DMA engine's start-up.

The right choice is workload- and hardware-dependent, which is the real lesson: **it's a knob to sweep, not a constant to assume**. The default (`HSA_ENABLE_SDMA=1`) is the right starting point; forcing blit costs up to ~25 % of halo time on the staged and GPU-Aware point-to-point paths. Turn on `HSA_ENABLE_SDMA_GANG=1` when the halo volume is large.

Next: [6. ROCm version & the SpMV API](#) →

6. ROCm version & the SpMV API

The transport and copy-engine chapters optimize **communication**. But the comm/compute breakdown from Chapter 3 has a second column — **compute** — and it turns out to be the dominant, and most surprising, part of the story at this problem size. This chapter is a worked example of using the same measurement discipline to chase down a compute-side regression, and of not being fooled by a plausible-sounding fix.

Sweeping ROCm versions

Because the binary must be rebuilt per toolchain, the sweep harness rebuilds `cg_gpu` under each module and runs one method with warm-up + repeats, all in one exclusive allocation (so results are directly comparable):

```
sbatch --export=ALL,VERSIONS="6.4.1 7.0.2 7.2.4 7.13.0",REPEATS=5,METHOD=isend \
submit_rocm_sweep.sbatch
```

Result: a compute regression at the 6.x → 7.x boundary

`isend`, 4 ranks, seed 12345, min of 5 runs (compute = solve – comm):

ROCm	solve (s)	comm (s)	compute (s)
6.4.1	0.0480	0.0178	0.0302 (baseline)
7.0.2	0.0717	0.0148	0.0569
7.2.4	0.0700	0.0141	0.0559
7.13.0	0.0713	0.0146	0.0567

Two things jump out:

- **Communication is as good or better on 7.x** (comm 0.018 → 0.014 s) — the MPI/UCX path did not regress.
- **Compute roughly doubled** (0.030 → 0.056 s) and is **flat across the whole 7.x line**, i.e. introduced at the 6.x→7.x boundary and not since recovered. Since compute now dominates, total solve time regressed ~1.5x.

This is only visible because the measurement splits comm from compute. A blind "solve time" comparison would have wrongly implicated the communication stack.

The plausible fix that isn't: `rocparse_v2_spmv`

On ROCm 7.x the classic `rocparse_spmv` is deprecated in favour of `rocparse_v2_spmv` (a persistent SpMV descriptor + an explicit one-time analysis stage). The obvious hypothesis: the regression is API/host overhead, and the new API will fix it. The examples let you test that hypothesis directly:

```
make SPMV_V2=1      # compile the rocparse_v2_spmv path (auto-falls back to v1 on ROCm 6.x)
# compare both paths across the 7.x line:
sbatch --export=ALL,SPMV="v1 v2",VERSIONS="6.4.1 7.0.2 7.2.4 7.13.0",REPEATS=5,METHOD=isend \
submit_rocm_sweep.sbatch
```

`SPMV_V2=1` also folds in a second micro-optimization applied to both paths: the dense-vector descriptors are created once and rebound with `rocparse_dnvec_set_values` each iteration instead of being created/destroyed per SpMV.

Result: v2 does not recover the regression

compute (s), min of 5 runs:

ROCm	v1 (rocparse_spmv)	v2 (rocparse_v2_spmv)
6.4.1	0.0302 (baseline)	— (v2 not available)
7.0.2	0.0569	0.0547
7.2.4	0.0559	0.0558
7.13.0	0.0567	0.0552

v2 improves 7.x compute by only **0-3 % (within noise)** and leaves the ~1.85x gap vs 6.4.1 intact. Since the per-call descriptor fix is now in the v1 numbers too and v1 is still slow, **the regression is in the rocSPARSE CSR SpMV**

compute kernel itself, not in the API surface or host/descriptor overhead. (Migrating to `rocsparse_v2_spmv` is still worthwhile for API longevity — v1 is deprecated — just not as a performance remedy.)

The methodology lesson: **a controlled experiment can disprove a plausible fix**. Because both paths were measured under identical conditions with the comm/compute split, the "it's the deprecated API" hypothesis was cleanly falsified. The next step to localize it is a one-iteration `rocprofv3` kernel trace on 6.4.1 vs 7.2.4 (Chapter 7).

Cross-toolchain: OpenMPI vs cray-mpich

The same solver was also built with **PrgEnv-amd / cray-mpich** (ROCm 7.0.3) and measured identically. `isend`, 4 ranks, same node class & matrix:

stack	ROCm	solve	comm	halo	dot allreduce	compute
OpenMPI 5.0.10	6.4.1	0.051	0.028	0.014	0.014	0.024
OpenMPI 5.0.10	7.2.4	0.111	0.020	—	0.003	0.091
cray-mpich 9.1.0	7.0.3	0.097	0.018	0.015	0.004	0.079

- **The MPI stack is not the differentiator here.** cray-mpich's GPU-Aware communication is on par with (a touch better than) GPU-Aware OpenMPI; the total-solve difference between rows is driven by the **ROCm compute version**, not the MPI implementation. ROCm 7.0.3 lands, as expected, between the 7.0.2 and 7.2.4 compute points.
- **The affinity cliff did not reproduce on Cray** (Chapter 3): an `--exclusive srun` already binds NUMA-locally, so explicit binding bought only ~3–8 %. Same solver, different launcher defaults.

Full details: [STUDY_REPORT.md](#) §5/§5.1 (OpenMPI) and [STUDY_REPORT_PrgEnv-amd.md](#) (cray-mpich).

Takeaways

- Always split comm vs compute before blaming the network — here the network was fine and the library kernel regressed.
- Rebuild per toolchain and compare in one allocation; binaries are not runtime-compatible across major ROCm versions anyway (a 7.x-linked binary against a 6.x runtime fails with an undefined HSA symbol).
- A new API is not automatically a faster API — measure it the same way you measure everything else.

Next: [7. The optimized configuration →](#)

7. The optimized configuration

You've now measured every axis. This chapter assembles them into a recommended setup, a decision checklist, and the directions worth pursuing next.

The optimization path at a glance

Performance progression as each optimization is applied (method = `isend`, 4 ranks, `Dubcova2.pm`, `CG_SEED=12345`, min of runs on one MI300A node):

Step	What changed	Config	Solve (s)	vs prev	vs naive GPU
0	CPU reference (baseline)	CG-CPU, 4 ranks	0.159	—	—
1	Naive GPU: default transport, no affinity	staged, unbound/non-exclusive	9.27	—	1×
2	Fix CPU/GPU affinity (exclusive + <code>gpu_bind.sh</code>)	staged, bound	0.148	63×	63×
3	Switch to GPU-Aware transport	<code>isend</code> , bound	0.051	2.9×	182×
4	Enable SDMA + gang copy engines	<code>isend</code> , bound, <code>HSA_ENABLE_SDMA_GANG=1</code>	0.045	1.13×	206×

The optimized GPU run (0.045 s) is ~206× faster than the naive GPU run and ~3.5× faster than the CPU reference — and **~90 % of that gain is affinity alone** (step 2). Steps 1–3 are ROCm 6.4.1; step 4 is 6.4.3.

Per-axis detail behind each step:

Step 2 — affinity (solve time, bound vs unbound):

method	unbound, not exclusive	--exclusive + <code>gpu_bind.sh</code>	speedup
staged	9.27 s	0.15 s	62×
isend	10.58 s	0.05 s	212×
rccl	7.39 s	0.05 s	148×
alltoallv	8.46 s	0.05 s	169×

Step 3 — transport (bound, ROCm 6.4.1):

method	solve	comm	halo exch	dot allreduce	compute	comm %
staged	0.1477	0.1243	0.1095	0.0148	0.0234	84 %
isend	0.0513	0.0282	0.0142	0.0140	0.0231	55 %
rccl	0.0505	0.0252	0.0160	0.0092	0.0253	50 %
alltoallv_staged	0.0823	0.0560	0.0422	0.0138	0.0263	68 %
alltoallv	0.0512	0.0250	0.0129	0.0121	0.0262	49 %

Step 4 — SDMA vs blit (ROCm 6.4.3, halo-exchange time):

method	blit	sdma	sdma_gang	best Δ vs blit
staged	0.1215	0.1116	0.1129	−8 %
isend	0.0154	0.0119	0.0113	−27 %
rccl	0.0143	0.0129	0.0129	−10 %
alltoallv_staged	0.0480	0.0418	0.0394	−18 %
alltoallv	0.0116	0.0140	0.0121	+21 %

Separate axis — ROCm version / SpMV API (compute time, `isend`). Not a speedup you "apply" but a version choice: staying on ROCm 6.4.x avoids a ~1.85× compute regression that `rocparse_v2_spmv` does not recover:

ROCm	v1 rocsparse_spmv	v2 rocsparse_v2_spmv
6.4.1	0.0302 (baseline)	—
7.0.2	0.0569	0.0547
7.2.4	0.0559	0.0558
7.13.0	0.0567	0.0552

Caveats: single node / single matrix / latency-bound regime; the cumulative path mixes ROCm 6.4.1 (steps 2–3) and 6.4.3 (step 4); for the overlap variants the `halo` figure is exposed wait, not raw wire time.

The recommended launch

For a distributed GPU CG solve on MI300A, in priority order:

1. **Exclusive allocation** that owns whole NUMA nodes (`sbatch --exclusive`).
2. **Per-rank GPU + NUMA-local CPU binding** (`gpu_bind.sh` or `set_affinity_mi300a.sh`, launched under `--bind-to none` / `--cpu-bind=none`). This is worth up to $\sim 100\times$ — nothing else comes close.
3. **A GPU-Aware transport** — `isend`, `alltoallv`, or `rccl`. Avoid the `staged` variants unless you have no GPU-Aware MPI; the staging copies cost $\sim 7\times$ the halo time.
4. **SDMA copy engines on** (the default), with `HSA_ENABLE_SDMA_GANG=1` if halo volume is large.
5. **The best available ROCm compute version** — on this workload ROCm 6.4.x compute is $\sim 1.85\times$ faster than 7.x; check whether that still holds on your version before locking it in.

```
cd CG-GPU
make                               # (or: make SPMV_V2=1 on ROCm 7.x for API longevity)

CG_SEED=12345 \
HSA_ENABLE_SDMA=1 HSA_ENABLE_SDMA_GANG=1 \
mpirun -n 4 --bind-to none ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm isend
```

Reproduce any of the studies end-to-end:

```
sbatch submit_cg_gpu_test.sbatch           # all 5 methods + convergence + timing
sbatch --export=ALL,REPEATS=5 submit_sdma_sweep.sbatch # SDMA vs blit, all methods
sbatch --export=ALL,SPMV="v1 v2",REPEATS=5 submit_rocm_sweep.sbatch # ROCm versions x SpMV API
sbatch --export=ALL,CG_SEED=12345 submit_cg_gpu_cray.sbatch # PrgEnv-amd / cray-mpich
```

Decision checklist: "which transport should I use?"

```
Do you have GPU-Aware MPI?
├ no → staged / alltoallv_staged (host-buffered) are your only correct options.
|     Expect halo to be dominated by D+H copies. Turn SDMA on.
└ yes → use a GPU-resident transport:
    ├── mostly neighbour exchange, few peers → isend
    ├── dense/irregular all-to-all pattern → alltoallv
    └─ want GPU-native, hipGraph-friendly, → rccl
        or plan to overlap on a side stream
(On one node over XGMI these are within noise – pick on ergonomics,
then measure on your real pattern/scale.)
```

Where the time actually goes — and what to optimize next

Once affinity is fixed and a GPU-Aware transport is chosen, the remaining per-iteration cost splits into three buckets. Attack them in ROI order:

A. Dot-product synchronization (highest ROI)

Each iteration does **two** `MPI_Allreduce`s, each gated by a full `hipDeviceSynchronize`. That's up to $\sim 27\%$ of solve on this problem and it is independent of the halo transport, so no amount of transport tuning touches it. Options:

- **Fuse the two dot products** (`r·r` and `p·Ap`) into a single allreduce where the algorithm allows.

- Adopt a **pipelined / communication-avoiding CG** (Ghysels & Vanroose pipelined CG, or s-step CG) to overlap the global reduction with the SpMV and cut the number of synchronization points.
- Replace `hipDeviceSynchronize` with **stream/event-scoped** sync so unrelated GPU work isn't stalled.

B. Kernel-launch overhead (small per-GPU problems)

At ~16k rows/GPU the loop is launch-latency bound. Capture the per-iteration kernel/BLAS sequence in a **HIP graph**, or fuse the rocBLAS `axpy/scal/dot` chain into custom kernels, to cut launch count.

C. SpMV / comm overlap

- In `rccl`, replace the full `hipDeviceSynchronize` after the gather with an **event + `hipStreamWaitEvent`** so only the RCCL stream waits.
- For the point-to-point variants, split the on-proc SpMV to better hide `MPI_Waitall` latency.

D. The compute-kernel regression (Chapter 6)

If you're on ROCm 7.x and compute-bound, the biggest single factor may be the rocSPARSE CSR SpMV regression. Localize it with a **one-iteration `rocprofv3` kernel trace** on 6.4.1 vs your version to identify the kernel whose duration grew, try `rocsparse_spmv_alg_csr_rowsplit` for this near-uniform-nnz matrix, and file an issue with the reproducer if confirmed.

Analysis directions to deepen the study

1. **Strong & weak scaling** — sweep 1→4→8 GPUs, then multi-node, to expose intra-node (XGMI) vs inter-node behaviour, where `rccl` and GPU-Aware `alltoallv` should start to diverge.
2. **Problem-size sweep** — larger matrices per GPU shift the balance from latency-bound to bandwidth/compute bound and shrink the exposed halo of the overlap variants; report a compute-bound regime to complement this latency-bound one.
3. **Raw vs exposed communication** — add non-overlapped timing or profiler traces to separate wire time from exposed wait for the overlap variants.
4. **Profiler-level attribution** — `rocprofv3` / `rocprofiler-systems` to attribute time to SpMV, dot, allreduce, and copies, and to confirm the NUMA-locality benefit at the hardware-counter level.
5. **Communication-volume model** — correlate measured halo time with actual per-rank ghost byte counts (from `form_comm`) to build a latency + bandwidth model and predict behaviour at scale.

The one-paragraph summary

The path to an optimized, comparable performance measurement on AMD multi-GPU nodes is: **bind first** (affinity dominates everything), **fix the seed and take warm minimums** (so runs are comparable), **split comm from compute** (so you optimize the right thing), then compare **transport** (use GPU-Aware MPI; the GPU-resident variants tie on one node), **copy engine** (SDMA on, gang for big halos), and **toolchain/version** (watch for compute-side regressions the network numbers won't reveal). The examples in this repo are instrumented and scripted so that each of these is a controlled, reproducible experiment rather than a guess.

← Back to the [tutorial index](#) · In-depth data: [STUDY_REPORT.md](#) · [STUDY_REPORT_PrgEnv-amd.md](#)